ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/385971459

Monte Carlo Graph Search and Transpositions in Monte Carlo Tree Search

Preprint · November 2024

DOI: 10.13140/RG.2.2.29303.43688

CITATIONS
0

READS
193

1 author:

Aaron Mulvey
Maastricht University

1 PUBLICATION **0** CITATIONS

SEE PROFILE

All content following this page was uploaded by Aaron Mulvey on 20 November 2024.

The user has requested enhancement of the downloaded file.

Monte Carlo Graph Search and Transpositions in Monte Carlo Tree Search

Aaron Mulvey Izquierdo
Department of Advanced Computing Sciences
Faculty of Science and Engineering
Maastricht University
Maastricht, The Netherlands

***Abstract*—This thesis explores the enhancement of Monte Carlo Tree Search through the incorporation of transposition tables and the transformation of its structure into a Directed Acyclic Graph, resulting in Monte Carlo Graph Search. Traditional MCTS constructs a search tree without accounting for state transpositions, where different sequences of actions lead to the same state, causing inefficiencies due to redundant node representations. This research compares two methods to address these inefficiencies: the use of transposition tables and the conversion of MCTS to MCGS. These approaches were tested within the Omega strategic board game, comparing their performance to conventional MCTS implementations. The results indicate that handling state transpositions significantly boosts the efficiency and effectiveness of MCTS, with MCGS showing promise for broader AI applications because of its inherent capability to manage state transpositions.**

***Index Terms*—Monte Carlo Tree Search, Monte Carlo Graph Search, transposition tables, board games, strategic decision-making, Directed Acyclic Graph.**

I. Introduction

Board games are commonly used as test domains to develop decision-making algorithms for Artificial Intelligence. Their implementation is straightforward due to the simplicity of their rules [1]. Search techniques such as *minimax* or *alpha-beta pruning* have been the standard approach in board game programs [2]. However, these algorithms require a proper evaluation function and a low branching factor [3]. An alternative has been the *Monte Carlo Tree Search (MCTS)*, which has been shown to be very effective in board games [2].

MCTS is a *best-first search* algorithm [4] that explores combinatorial spaces represented as a tree data structure, where nodes indicate states or problem configurations, and edges represent transitions or actions [5]. Using a tree data structure ensures that there are no cycles or self-loops, which is necessary for the convergence of the algorithm. Although this representation is generally effective, the possibility that different paths may reach the same state (known as transpositions) can reduce the efficiency of MCTS. MCTS creates multiple nodes for the same state, which can be inefficient in problems with frequent transpositions. This leads to the following problem statement:

- **How can we improve the efficiency and applicability of MCTS in strategic decision-making in domains where transpositions occur?**

There are two approaches to handling transpositions: one is the use of an additional data structure called *transposition tables* [6], and the other is to modify the tree structure of MCTS into a *direct acyclic graph DAG* to achieve *MCGS* [10].

This research closely examines two approaches in MCTS, specifically for the complex game Omega. We aim to determine whether including transpositions is more effective than the traditional methods of MCGS. The goal is to enhance our understanding of how to improve decision-making in complex games, building on previous research by Childs, Brodeur, and Kocsis [11] on the benefits of transpositions and move groups in MCTS , and work by Czech, Korus, and Kersting [14], who demonstrated how directed acyclic graphs can make MCTS more efficient. The three research questions to address the problem statement are:

- How does the integration of transposition tables impact the computational efficiency and decision-making accuracy of MCTS in complex game environments?
- In what ways does transitioning from a traditional tree-based structure to a DAG in MCGS enhance the handling of state transpositions?
- What are the comparative advantages of MCGS over traditional MCTS in terms of performance metrics such as win rates and search efficiency in abstract board games?

This thesis is structured as follows. Section II begins with an explanation of the vanilla MCTS algorithm. Section III reviews current advancements integrating transposition tables with MCTS. In Section IV, the MCGS algorithm is detailed. Subsequent sections systematically present the empirical work undertaken: Section V introduces the board game implemented for the conduction of the experiments. Section VI outlines the experimental setup, Section VII presents the findings, Section VIII interprets these results, and finally, Section IX concludes the thesis by summarizing key insights and discussing future research directions.

II. Monte Carlo Tree Search

MCTS is an algorithm that iteratively builds a search tree until a predefined stopping condition is met [7]. It then selects

This thesis was prepared in partial fulfilment of the requirements for the Degree of Bachelor of Science in Data Science and Artificial Intelligence, Maastricht University. Supervisors: Prof. Dr. Mark Winands and Dr. Matúš Mihalák.

the optimal child node from the root node based on the results of its search. Each iteration of MCTS comprises four sequential steps [8]:

1) *Selection*: Beginning at the root node, the algorithm traverses the section of the tree currently stored in memory. At each level, it selects the next node based on a specific *tree policy*. This process continues until it reaches a node that has not yet been fully explored [5], [7].
2) *Expansion*: During this phase, the algorithm expands the tree by adding at least one new node, which represents a possible action from the current state.
3) *Simulation*: From this new node, the algorithm simulates a playthrough to a terminal state. The outcome of this simulation, typically some form of score or payoff, is then recorded.
4) *Backpropagation*: Finally, the simulation result is propagated back through the tree from the node of termination to the root node. During this backpropagation, the statistics in each node along the path are updated to reflect the new information.

These steps are repeated until the stopping criteria is satisfied. At this point, the algorithm selects the best-performing child node of the root based on the accumulated data.

The tree policy during the selection phase aims to maintain a balance between exploration and exploitation. A common algorithm used for this purpose is the Upper Confidence Bounds applied to Trees (UCT) [9], which selects the action that maximizes the following formula:

$$UCT(s) = \arg\max_{a \in A(s)} \left\{ Q_{s,a} + C\sqrt{\frac{\ln N_s}{N_{s,a}}} \right\} \tag{1}$$

$$Q_{s,a} = \frac{R_{s,a}}{N_{s,a}} \tag{2}$$

$$N_s = \sum_{a \in A(s)} N_{s,a} \tag{3}$$

Here, $A(s)$ represents the set of available actions or nodes from state s, and $Q_{s,a}$ is the average result of executing action a in state s. This average is calculated as the total rewards $R_{s,a}$ obtained from action a in state s, divided by the number of times that action has been simulated $N_{s,a}$. The constant C serves as a tuning parameter to balance exploration and exploitation. Finally, N_s denotes the total number of times the state s has been visited during simulations [5].

III. Transposition Tables

A transposition table is a data structure used to uniquely store information about transposed states from a tree. This enables the sharing of information about identical states across different nodes, thereby allowing the search tree to be represented as a DAG. To create an entry in a transposition table, the first requirement is to obtain a hash value that is unique for each state. The most common method to derive this value in game playing is through Zobrist hashing [12]. This hashing function generates a random number for each possible combination of position and piece on a game board. A 64-bit hash value is then computed using the XOR operation on these numbers [6]. Once the hash value is obtained, it is added to the table. To optimize the memory usage in the table, it will have a fixed size of 2^n bits where n represents the first n bits of the hash value. As illustrated in Figure 1, this configuration results in the hash value now being composed of a hash index, which indicates the position of an entry in the table, and a hash key, which is used to differentiate between entries that share the same hash index. One issue with this specific representation of transposition tables is that two distinct states can end up sharing the same hash index. A common solution to this problem is to link each entry to the next one, thereby allowing each table entry to function like a linked list, where all elements in the list share the same hash index but have different hash keys.

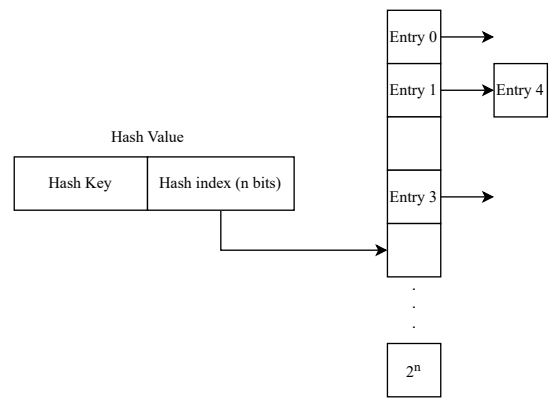

Fig. 1. Transposition Table.

Transposition tables have been used in conjunction with the Alpha-Beta pruning algorithm in games such as chess [6], [10]. They have also been effectively combined with the Monte Carlo Tree Search (MCTS) algorithm in the game of Go, showing improvements over traditional MCTS methods. Childs et al. [11] introduced three ways to utilize transposition tables in combination with the MCTS by modifying the UCT algorithm making four variants, (UCT0, UCT1, UCT2 and UCT3).

1) Implementation: The implementation of the transposition tables was done as follows: The transposition table is represented by an array of table entries. Each table entry contains the hash value of the state, relevant statistics of the transposed state, that is, the number of times the states was simulated N_s and the number of victories obtained R_s. Additionally, each entry contains two linked lists. The first linked list stores the hash values of predecessor states, and the second linked list contains the hash values of successor states. Finally, each entry has a pointer to another entry. This is to ensure that in the case of hash collisions, the entry is not overwritten.

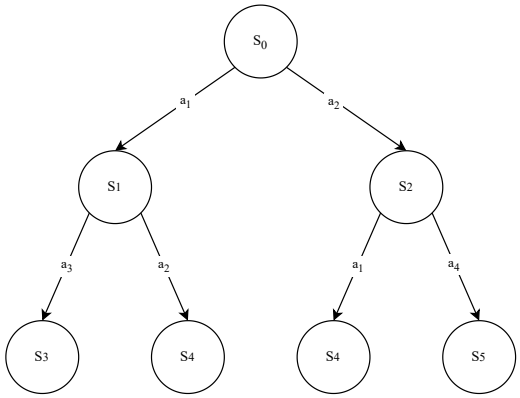

Fig. 2. MCTS search tree for single player game

A. UCT0

The classic MCTS, called UCT0 by Childs et al. [11], does not use transposition tables. Consequently, the UCT value is computed using the standard formula shown in equation (1). As depicted in Figure 2, when an action from node S_1 to S_4 is selected, only the statistics pertaining to that particular transition are utilized. Additionally, during the backpropagation phase, the $Q_{s,a}$ values of the nodes encountered during the simulation phase are updated.

It is assumed that for any state s in the tree, the number of times s has been simulated, N_s, is strictly greater or equal to the number of times the action $a \in A(s)$ has been simulated $N_{s,a}$, as can be seen in equation (3). This is because in each iteration of the MCTS, a single node is expanded and simulated, and during the backpropagation phase, all nodes along the selected path are updated.

However, when transposition tables are used, the aforementioned no longer holds. If a transposed node is encountered during the backpropagation phase, only the transposed node—that is present in multiple paths—is updated, but not all paths. Consequently, it is now possible for the node to have fewer number of simulations as some of its children.

B. UCT1

In this variant, transposition tables are used without modifying the selection formula. If a transposition is identified during the selection phase, the relevant information for selection is accumulated regardless of its location in the tree. During the backpropagation phase, all nodes along the traversed path are updated. If any of these nodes is a transposed node, this implies that the node will be updated across all paths in which it is present. As can be seen in Figure 2, from the root node, whether actions a_1 or a_2 are taken, the resulting state S_4 remains the same, since the order in which the actions are taken does not affect the final state. Thus, the number of simulations of S_4 will be equal to the sum of the simulations of its parents, i.e., $N_{s_1,a_2} + N_{s_2,a_1}$. If the UCT1 formula is applied from state S_0, the result would be exactly the same as if the UCT0 formula were applied, because the states $S1$ and $S2$ are not affected by the transpositions. However, if the UCT1 formula is applied at state $S1$ or $S2$, in this case, it would utilize the information from the transposed node $S4$.

C. UCT2

As observed, UCT1 cannot fully exploit transpositions, as mentioned previously; from node S_0 it would be indifferent to use either UCT1 or UCT0. To address this situation, UCT2 takes a further step by better utilizing the statistics of transposed nodes than the previous variant. This is achieved by replacing the use of $Q_{s,a}$ with $Q_{g(s,a)}$, where $g(s,a)$ denotes the position resulting from the execution of action a in state s. This change is formalized in the following equation:

$$UCT2(s) = \arg\max_{a \in A(s)} \left\{ Q_{g(s,a)} + C\sqrt{\frac{\ln N_s}{N_{s,a}}} \right\} \tag{4}$$

$$Q_{g(s,a)} = \sum_{b \in A(s)} \frac{R_{s,b}}{N_{s,b}} \tag{5}$$

In this way, if UCT2 is applied to state S_0 and the action a_1 is being evaluated, $g(s_0, a_1)$ will result in state S_1. Therefore:

$$Q_{g(s_0,a_1)} = \frac{R_{s_1,a_3} + R_{s_1,a_2}}{N_{s_1,a_3} + N_{s_1,a_2}} \tag{6}$$

Thus, in this case, information from the transposed node S_4 will be included. Childs et al. [11] pointed out that using $N_{g(s,a)}$ in the exploration term could lead to inaccuracies if the optimal move is approached through different paths.

D. UCT3

UCT2 only resolves the issue of UCT1 at a depth of one. If more transpositions occur at depths greater than this, the use of UCT2 and UCT0 could become equivalent, as was previously observed with UCT1.

To sove this UCT3 follows the same idea as UCT2 using $Q_{g(s,a)}$, the difference is that this variant refines the action value more by using the information from each child recursively. The simplest way to implement this approach is by modifying the backpropagation phase to update all paths present in the simulation. In this way the equality of the equation (3) will always hold.

IV. Monte Carlo Graph Search

Cazenave et al. [10] introduced a parametric formula for the UCT algorithm using a DAG structure instead of a tree, marking a significant shift from traditional tree-based models to more complex graph structures. The introduced formula, called the Upper Confidence Bound for Rooted Directed Acyclic Graphs (UCD) as seen in equation (7), includes three parameters d_1, d_2, and d_3 that belong to $d_i \in \mathbb{Z}^+$. The values of d_1, d_2, and d_3 represent the depth from which transposition information is considered and are usually chosen as 0, 1, or ∞. The parameter configuration $d = (0, 0, 0)$ makes the behavior of the MCGS comparable to UCT1. On the other hand, to manage transpositions as in UCT2, the configuration

$d = (1, 0, 0)$ is used, and for UCT3, $d = (\infty, 0, 0)$ is employed.

$$UCD(s) = \arg\max_{a \in A(s)} \left\{ Q_{d_1(s,a)} + C\sqrt{\frac{\ln N_{d_2(s)}}{N_{d_3(s,a)}}} \right\} \quad (7)$$

As can be seen in Figure 3, this approach allowed for multiple parent nodes, essentially capturing transpositions directly within the DAG, thereby eliminating the need for auxiliary data structures like transposition tables. Results from game simulations were stored on the edges, enhancing the ability to detect transposed states without additional overhead [10]. Building on this foundational work, Czech et al. [14] demonstrated the efficacy of integrating MCGS with AlphaZero, leading to notable improvements in both performance and efficiency when compared to conventional MCTS techniques. This evolution from MCTS to MCGS indicates a promising direction for the field, leveraging the inherent strengths of DAGs for sophisticated state management in strategic game decision-making.

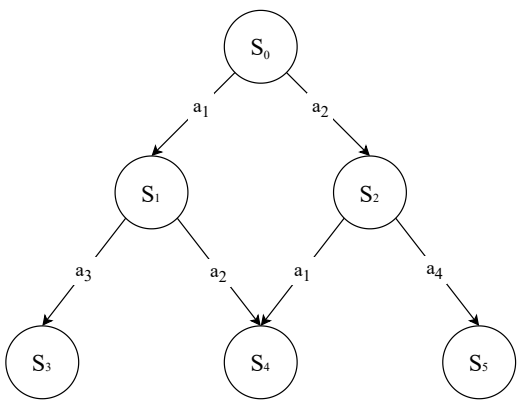

Fig. 3. MCGS for single player game

As mentioned before, the MCGS algorithm can be implemented in various ways, depending on the parameter configuration used. For a fair comparison with the previously discussed algorithms, it was decided to implement three variants that handle transpositions similarly to UCT1, UCT2, and UCT3. These variants are designated as UCD1, which has a configuration of $d = (0, 0, 0)$, UCD2 with $d = (1, 0, 0)$, and UCD3 with $d = (\infty, 0, 0)$, respectively. This naming convention facilitates the easy identification of the algorithms, where those using transposition tables are designated as UCT and those utilizing the DAG structure as UCD. The number accompanying each algorithm's name indicates the manner in which transpositions are handled.

As mentioned before, the MCGS algorithm can be implemented in various ways, depending on the parameter configuration used. For a fair comparison with the previously discussed algorithms, it was decided to implement three variants that handle transpositions similarly to UCT1, UCT2, and UCT3. These variants are designated as UCD1, which would have a configuration of $d = (0, 0, 0)$, UCD2 with $d = (1, 0, 0)$, and UCD3 with $d = (\infty, 0, 0)$, respectively. This naming convention is used for the easy identification of the algorithms used, where the algorithms that use transposition tables are those of UCT and those that use the DAG structure are those of UCD, and the way transpositions are handled is indicated by the number accompanying the name of the algorithm.

1) UCD1: This variant is equivalent to UCT1, with the key difference that each time a transposed state is encountered, instead of generating another node, an additional edge is added to the transposed node as depicted in algorithm 1. From this point, the tree structure transitions into a graph. One of the challenges of working with a graph is that during backpropagation, it is unclear which path was followed during the tree descent. Therefore, to perform backpropagation correctly, it is necessary to store the followed path in a stack to update the correct path.

Algorithm 1: Expansion for MCGS

```
Input: Node p
Output: Node c
Node c;
State s ← p.getState();
if s is not transposed then
    c ← new Node(s);
    graph.put(s, c);
c.add(p);
return c;
```

2) UCD2: In the case of UCD2, the selection and expansion processes are identical to UCD1. The only difference arises in the backpropagation phase. If the node is terminal, both the node and all its parent nodes are updated. If it is an internal node, only the parent nodes are updated. This differentiation is necessary for the correct updating of nodes; otherwise, some nodes would be updated multiple times.

Algorithm 2: Update All

```
Input: Node n, double w, int d
if d = 0 then
    n.update(w);
    foreach a in n.parents do
        a.update(w);
else
    n.update(w);
    foreach a in n.parents do
        Update All(a, w, d - 1);
```

3) UCD3: Finally, the UCD3 variant is similar to the previously mentioned variants, with the key difference that in this variant, the path is not used for updating during the backpropagation phase. Instead, all paths are updated.

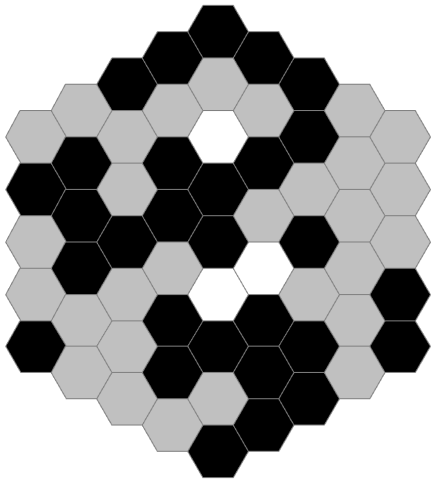

Fig. 4. Board of a ended Omega game

V. Omega Board Game

As previously mentioned, to compare the different algorithms, the Omega game is used. The Omega board game is a multiplayer game for 2 to 4 players played on a hexagonal board. In each turn, players place one piece of their color and another of their opponents' color. When there are not enough pieces to complete a full round of turns, the game ends. Once the game ends, to determine the winner, the points of each player are counted. The player with the most points will be the winner; if two players have the same score, it is considered a tie. To determine a player's score, the sizes of the player groups of pieces are multiplied. A group is considered to be all pieces of the same color that are connected, with the minimum group size being 1. As shown in Figure 4, there are three free cells left. For a 2-player game, 4 pieces are placed each round, so the game has ended. The first player's score is $12 \times 10 \times 7 = 840$, while the second player's score is $16 \times 8 \times 2 = 256$. Therefore, the winner is the first player.

VI. Experimental Set Up

The following section introduces how the experiments in this specific work were conducted. Initially, in VI-A, the configuration common to all algorithms for the experiments is detailed. In VI-B, exploration of the game configuration is provided. Subsequently, the experimental procedure and the equipment used for the experiments are detailed in VI-C. Lastly, in VI-D, the data analysis methodology for the experiments is described.

A. Configuration of the Implemented Algorithms

To conduct the experiments, the algorithms were configured as follows: The constant $C = 0.4$ was established to balance exploration and exploitation during the selection phase. In the expansion phase, only one new node was added in each iteration of the search process. The search process was halted upon reaching a predetermined number of simulations or when a specified time limit was reached.

Regarding final selection, the policy adopted was to select the child node with the highest performance measured by $R_{s,a}/N_{s,a}$. The play out strategy was purely random, and all pseudorandom numbers generated during the simulation phase used the same linear congruential generator (LCG) and started from an identical seed to ensure consistency in the simulations.

B. Game Configuration

The number of players was set to two for all games, using a board of size five, resulting in a total of 91 available squares at the start of each game. Each turn in the game was treated individually in terms of the search, with a separate search performed for each piece placed, rather than one per turn. This implies that in each turn, two searches were conducted, one for each player.

C. Experimental Procedure

The algorithms competed in a round-robin format, where each match was conducted 100 times, alternating the positions of the players to avoid initial play advantages. This procedure was repeated under two conditions: the first with a stop after 10,000 iterations, and the second with a time limit of 1000 ms. In each match, data on the number of victories x and the total number of games n were collected. The time spent on selection and backpropagation for each algorithm was also recorded. Data on the number of transposed states in each round were also collected.

When collecting the time it takes in the search phase of the algorithms, a simple experiment was also conducted to increase the number of iterations in the search and collect the data on the average time needed to complete the search from the initial position. This experiment was carried out for the UCT3 and UCD3 algorithms as they are the most computationally intensive due to the backpropagation phase, and UCT0 was used as a baseline. The experiments were conducted on a 2020 MacBook Air with an Apple M1 CPU and 8GB of RAM. The programming language used for both the implementation of the games and algorithms and for conducting the experiments was Java 17.

D. Statistical Analysis

The scoring rate w was defined as $w = x/n$, indicating the probability of winning against an opponent [16]. This index ranges from 0 to 1. The standard error of the scoring rate $s(w)$ was calculated as can be seen in (8), and confidence intervals for the probability of victory were established using a critical value $z_{95\%} = 1.96$ from the normal distribution $N(0, 1)$ as can be seen in (9), corresponding to a 95% confidence level. This level ensures that if the experiment were repeated under the same conditions, the confidence interval would include the true probability of winning 95% of the time.

$$s(w) = \frac{\sqrt{w \cdot (1 - w)}}{\sqrt{n}} \tag{8}$$

$$w \pm z_{\%} \cdot s(w) \tag{9}$$

VII. Results

In the following section, the results obtained from the experiments previously described will be shown. The first part VII-A displays the results from the round-robin experiments and also presents data on the number of transposed nodes to further contextualize the results obtained. The second part VII-B shows the search time data of various algorithms to measure the temporal efficiency of using transpositions as the number of simulations increases.

A. Performance Comparison

The algorithms were tested under two primary conditions: a fixed number of iterations and a fixed time limit. The results highlighted distinct performance differences between the MCTS and MCGS methods. Tables I and II present the win rates, expressed as percentages with confidence intervals, for all algorithm versions in a round-robin tournament under both conditions.

The data revealed that UCT3 and UCD3, which manage transpositions through more sophisticated mechanisms, generally outperformed the simpler versions. Notably, UCD3 achieved the highest win rates in most comparisons. While the difference compared to UCT3 may not be statistically significant, this suggests that utilizing graph-based methods within the MCGS framework to handle transposed states could be a viable alternative to transposition tables without sacrificing performance.

To ensure the reliability of results, standard errors and confidence intervals were employed. This statistical analysis confirmed that both UCT3 and UCD3, which use more advanced methods for handling transpositions, consistently outperformed the simpler versions. On average, 18.34% $\pm$ 3.93% of the states were transpositions in the fixed number of iterations and 20.22% $\pm$ 6.61% in the time-limited scenarios. This highlights the frequency of transpositions and underscores the importance of effective handling mechanisms in these algorithms, especially in complex games like Omega where numerous paths can lead to identical states.

The improvements of UCD3 were particularly pronounced compared to the basic UCT0 model, under both fixed iteration and time-limited tests, as shown in Tables I and II.

The observed improvements of UCT3 over UCT0 further emphasize the benefits of advanced transposition management. These findings not only validate the efficacy of advanced MCTS and MCGS methods in managing transpositios but also showcase their potential for broader application in domains that necessitate handling large and complex state spaces.

B. Efficiency of Transpositions

In this subsection, the additional time when looking at transpositions is going to be investigated. Figure 5 shows the mean search time to complete a given number of iterations for UCT3, UCD3, and UCT0. While the UCT3 and UCD3 methods generally showed better performance in win rates across both fixed iterations and time-limited configurations, the overhead of updating all paths leading to a transposed node results in additional overhead compared to UCT0.

The overhead of these two algorithms increases as the number of transposed states and the search depth increases. As can be observed in Figure 5, when the iterations are below 10,000, the difference between the three algorithms is not very noticeable, but as the number of iterations increases, UCT0 starts to become more efficient than UCT3 and UCD3. It is also noticeable that UCD3 is slightly more efficient than UCT3, but the difference is not as significant as with UCT0.

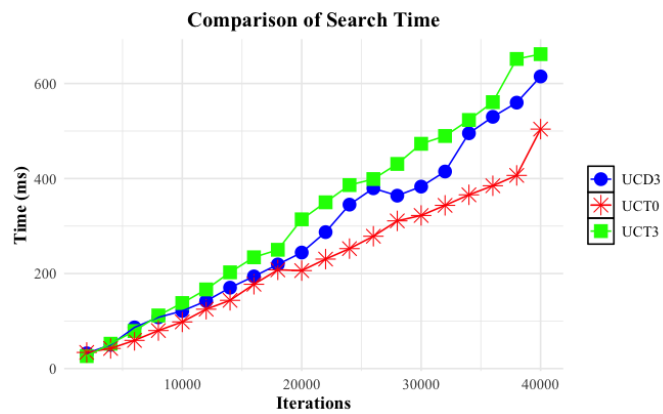

Fig. 5. Comparison of search time

VIII. Discussion

The experiments conducted for this thesis provide significant insights into the performance and efficiency of various MCTS and MCGS algorithms when applied to strategic decision-making in games such as Omega. This section discusses the key findings from the empirical studies, their implications, and the potential avenues for future research.

A. Effectiveness of Transposition Handling

The effectiveness of these techniques is supported by prior research. In the game of Go, Childs et al. [11] demonstrated that UCT1 and UCT2 algorithms improved the performance of UCT0, and that UCT3 provided significant further improvement when selection and simulation time dominated backpropagation time. These findings are consistent with the results shown in Tables I and II. Subsequently, Cazenave [10] introduced a way to work with transpositions without the need for transposition tables, generalizing the ideas proposed by Childs et al. [11] using graphs. This approach allowed for the adaptation of graphs to UCT1, UCT2, and UCT3 variants under a unified parametric formula. However, no comparison was made between UCD and UCT. In the work of Czech, Korus, and Kersting [14], MCGS was used in the context of chess alongside the AlphaZero algorithm, demonstrating that the use of graphs reduced redundant neural network evaluations.

TABLE I
ROUND ROBIN WITH 10,000 ITERATIONS

	UCT0	UCT1	UCT2	UCT3	UCD1	UCD2	UCD3
UCT0	–	48 ± 9.8	48 ± 9.8	39 ± 9.6	47 ± 9.8	48 ± 9.8	34 ± 9.2
UCT1	52 ± 9.8	–	50 ± 9.8	45 ± 9.7	49 ± 9.8	50 ± 9.8	38 ± 9.5
UCT2	52 ± 9.8	50 ± 9.8	–	43 ± 9.7	54 ± 9.8	46 ± 9.8	34 ± 9.2
UCT3	59 ± 9.6	55 ± 9.8	57 ± 9.7	–	66 ± 9.3	57 ± 9.7	48 ± 9.8
UCD1	53 ± 9.8	51 ± 9.8	46 ± 9.8	34 ± 9.3	–	52 ± 9.8	34 ± 9.2
UCD2	52 ± 9.8	50 ± 9.8	54 ± 9.8	43 ± 9.7	48 ± 9.8	–	41 ± 9.6
UCD3	66 ± 9.3	62 ± 9.5	67 ± 9.2	52 ± 9.8	66 ± 9.3	59 ± 9.6	–

Algorithm	Mean
UCT0	44.00
UCT1	47.33
UCT2	46.50
UCT3	57.00
UCD1	45.00
UCD2	48.00
UCD3	62.00

TABLE II
ROUND ROBIN WITH 1000 MS TIME LIMIT

	UCT0	UCT1	UCT2	UCT3	UCD1	UCD2	UCD3
UCT0	–	53 ± 9.8	45 ± 9.8	41 ± 9.6	41 ± 9.6	47 ± 9.8	38 ± 9.8
UCT1	47 ± 9.8	–	47 ± 9.8	46 ± 9.8	49 ± 9.8	47 ± 9.8	35 ± 9.3
UCT2	55 ± 9.8	53 ± 9.8	–	43 ± 9.7	51 ± 9.8	47 ± 9.8	32 ± 9.1
UCT3	59 ± 9.6	54 ± 9.8	57 ± 9.7	–	66 ± 9.3	46 ± 9.8	42 ± 9.7
UCD1	59 ± 9.6	51 ± 9.8	49 ± 9.8	34 ± 9.3	–	45 ± 9.8	32 ± 9.1
UCD2	53 ± 9.8	53 ± 9.8	53 ± 9.8	54 ± 9.8	55 ± 9.8	–	37 ± 9.5
UCD3	62 ± 9.6	65 ± 9.3	68 ± 9.1	58 ± 9.7	68 ± 9.1	63 ± 9.5	–

Algorithm	Mean
UCT0	44.17
UCT1	45.17
UCT2	46.83
UCT3	54.00
UCD1	45.00
UCD2	50.83
UCD3	64.00

One of the main objectives of this work was to evaluate the differences between these two ways of using transpositions by comparing them. Since the use of graphs eliminates the need for additional data structures and also simplifies the management of transpositions, the results clearly demonstrate that UCT3 and UCD3, both employing advanced methods for managing transpositions, consistently outperformed simpler MCTS and MCGS configurations. This improvement in performance underscores the value of integrating sophisticated transposition handling mechanisms, particularly in complex games where identical game states can be reached through different sequences of moves.

Furthermore, the UCT1, UCT2, UCD1, and UCD2 variants slightly outperformed the MCTS version that does not utilize transposition handling at all. This suggests that graph-based algorithms can effectively replace traditional transposition table methods without sacrificing performance, potentially enhancing algorithmic efficiency by eliminating the need to create unnecessary nodes or utilize an additional data structure specifically for transposition handling.

Lastly, the second experiment showed interesting data on the efficiency of UCT3 and UCD3, as it can be observed that UCT3 requires slightly more time for completing a given number of iterations compared with UCD3. This result is consistent with the results shown in Tables I and II since, although there is no significant difference, it can be observed that both UCT3 and UCD3 have a higher win percentage compared to UCT0 when the stopping condition is the number of iterations than when it is time.

IX. CONCLUSION & FUTURE RESEARCH

This thesis has enhanced the Monte Carlo Tree Search (MCTS) algorithm by integrating transposition tables and adopting a Directed Acyclic Graph (DAG) structure, termed Monte Carlo Graph Search (MCGS). Applied within the strategic board game Omega, these modifications have provided substantial improvements in managing transpositions, thus enhancing both the efficiency and strategic capabilities of the algorithms.

1) **Efficiency and Performance Enhancements:** Advanced transposition handling, particularly with the UCT3 and UCD3 algorithms, has significantly improved efficiency and decision-making in complex games such as Omega, markedly outperforming vanilla MCTS. These results highlight the critical role of sophisticated transposition management in reducing computational redundancies and enhancing strategic outcomes.
2) **Validation of Graph-Based Approaches:** The implementation of MCGS confirmed that graph-based structures effectively manage state transpositions without additional data structures like transposition tables, thereby simplifying the algorithmic framework and potentially boosting computational efficiency in games featuring complex state spaces.

These findings underscore the potential for graph-based search algorithms in applications involving large, complex state spaces, suggesting further exploration in fields such as robotics, automated planning, and even non-gaming areas like logistics and healthcare [13].

Future research could develop a hybrid backpropagation phase in MCGS, optimizing updates across all paths only when necessary, to enhance decision-making speed without compromising depth of analysis.

REFERENCES

[1] É. Beaudry, F. Bisson, S. Chamberland, and F. Kabanza (2010). "Using Markov decision theory to provide a fair challenge in a roll-and-move board game," in *Proceedings of the 2010 IEEE Confer-*

ence on Computational Intelligence and Games, pp. 1-8, IEEE, doi: 10.1109/ITW.2010.5593380.

[2] M. H. M. Winands (2017). "Monte-Carlo Tree Search in Board Games," in *Handbook of Digital Games and Entertainment Technologies*, pp. 47-76, Springer, doi: 10.1007/978-981-4560-50-4_27.

[3] G. Chaslot, S. Bakkes, I. Szita, and P. Spronck (2008). "Monte-Carlo Tree Search: A New Framework for Game AI," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, AAAI, doi: 10.1609/aiide.v4i1.18700.

[4] R. Coulom (2006). "Efficient Selectivity and Backup Operators in Monte Carlo Tree Search," in *International Conference on Computers and Games*, pp. 72–83, Springer.

[5] M. Świechowski, K. Godlewski, B. Sawicki, et al. (2023). "Monte Carlo Tree Search: a review of recent modifications and applications," *Artificial Intelligence Review*, vol. 56, pp. 2497–2562, doi: 10.1007/s10462-022-10228-y.

[6] D. M. Breuker (1998). "Memory versus Search in Games," Proefschrift (Doctoral Thesis), Universiteit Maastricht, The Netherlands.

[7] C. Browne, E. Powley, D. Whitehouse, S. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A Survey of Monte Carlo Tree Search Methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012, doi: 10.1109/TCIAIG.2012.2186810.

[8] G. M. J. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, and B. Bouzy (2008). "Progressive Strategies for Monte-Carlo Tree Search," *New Mathematics and Natural Computation*, Vol. 4, No. 3, pp. 343–357.

[9] L. Kocsis and C. Szepesvári (2006). "Bandit based Monte-Carlo planning," in *Proceedings of the 17th European Conference on Machine Learning*, pp. 282–293, Springer-Verlag, Berlin, Heidelberg, ECML'06.

[10] T. Cazenave, J. Méhat, and A. Saffidine (2012). "UCD: Upper confidence bound for rooted directed acyclic graphs," *Knowledge-Based Systems*, vol. 34, pp. 26-33, doi: 10.1016/j.knosys.2011.11.014.

[11] B. E. Childs, J. H. Brodeur, and L. Kocsis (2008). "Transpositions and Move Groups in Monte Carlo Tree Search," *IEEE Symposium on Computational Intelligence and Games*, IEEE.

[12] A. L. Zobrist (1970). "A New Hashing Method with Applications for Game Playing," Technical Report 88, Computer Science Department, The University of Wisconsin, Reprinted (1990) in *ICCA Journal*, Vol. 13, No. 2, pp. 69–73.

[13] C. F. Sironi (2019). *Monte-Carlo Tree Search for Artificial General Intelligence in Games*, Dissertation, Maastricht University, The Netherlands.

[14] J. Czech, P. Korus, K. Kersting (2020). "Monte-Carlo Graph Search for AlphaZero," *Proceedings of the AAAI Conference on Artificial Intelligence*, Association for the Advancement of Artificial Intelligence, pp. 1–9

[15] Y. Björnsson and H. Finnsson (2009). "CadiaPlayer: A Simulation-Based General Game Player," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 1, pp. 4–15, doi: 10.1109/TCIAIG.2009.2018702.

[16] E. A. Heinz (2000). "Self-Play Experiments in Computer Chess Revisited," *Advances in Computer Games*, pp. 74–75, Maastricht, The Netherlands.

View publication stats